

RELATÓRIO TÉCNICO DE AUDITORIA

WHEngenharia — Sistema ERP



ALTO RISCO
Classificação Final do Sistema

Data da Auditoria:	Abril de 2026
Classificação:	Confidencial
Repositório:	MarqSolucoes/REPO-TEMPORARIO
Escopo:	Frontend (Vue.js 2) + Backend (ASP.NET Core) + Banco de Dados (SQL Server)

Análise independente e crítica do código-fonte. Abril de 2026.

SUMÁRIO

1.	Resumo Executivo
2.	Visão Geral da Arquitetura
3.	Análise do Frontend
4.	Análise do Backend
5.	Banco de Dados
6.	Segurança
7.	Performance
8.	DevOps e Processo
9.	Problemas Críticos
10.	Melhorias Recomendadas
11.	Classificação Final

1. Resumo Executivo (não técnico)

! Estado Geral do Sistema

O sistema está em **estado crítico de risco**. Apresenta falhas graves de segurança que expõem credenciais de acesso ao banco de dados e senhas de e-mail no repositório.

Principais Riscos para a Diretoria

! Credenciais de produção no repositório

Qualquer pessoa com acesso ao repositório pode conectar ao banco de dados de produção agora.

! Empresa refém do desenvolvedor atual

Infraestrutura em conta pessoal, sem documentação de deploy. Uma saída do fornecedor compromete a continuidade do sistema.

! Bug confirmado no Fluxo de Caixa

Duplicação de valores em relatórios financeiros.

! Zero testes automatizados

Qualquer mudança pode quebrar funcionalidades sem que ninguém perceba.

2. Visão Geral da Arquitetura

Backend

- **Tecnologia:** ASP.NET Core (C#), Entity Framework Core, SQL Server
- **Estrutura:** 3 camadas — Controllers → Servicos → Repositorios → SQL
- **Problema:** **Serviços instanciados com new** dentro dos controllers, eliminando DI

Frontend

- **Tecnologia:** Vue.js 2 (**EOL desde dez/2023**), Vuex, Axios, Bootstrap-Vue
- **Problema:** `api.service.js` com 102 KB concentra todas as chamadas HTTP

Camada	Componente	Observação
Controller (31)	Recebe HTTP	Instancia com new — viola DI
Serviço (29)	Regras de negócio	Mistura PDF/Excel/negócio
Repositório (29)	Acesso a dados (EF Core)	Contém regras de negócio
SQL (Context)	ORM + schema	Adequado

3. Análise do Frontend

Hash de Senha no Frontend (SHA-256)

O frontend usa `js-sha256` para converter a senha antes do envio ao backend — tanto no login quanto no cadastro e alteração de senha. Isso evita o envio da senha em texto puro, mas apresenta limitações importantes:

```
// Login.vue:122
const senha = sha256(String(this.$refs.senha.value));

// Usuario.vue:1100
Senha: sha256(String(this.usuario_Senha)),
```

- **Sem salt:** senhas iguais produzem hashes idênticos — vulnerável a rainbow tables
- **SHA-256 é rápido:** não foi projetado para hashing de senhas. `bcrypt`/`Argon2` são preferidos
- **Hash client-side = credencial efetiva:** interceptar o hash equivale a capturar a senha do ponto de vista do servidor
- **Recomendação:** enviar senha via HTTPS e aplicar `bcrypt` no backend; remover `js-sha256` do frontend

Organização e Performance

- `Vue.js 2` em EOL desde dezembro de 2023 — sem patches de segurança
- `api.service.js` com 102 KB concentra todos os endpoints sem separação por domínio
- `Get()` sem paginação carrega toda a tabela de materiais sem limite
- `BuscaMaterialPorTexto` executa 2 queries separadas e faz dedução em memória

4. Análise do Backend

Injeção de Dependência — violada em todos os Controllers

```
// Padrão incorreto em todos os 31 controllers:  
_servicoPedidoCompra = new ServicoPedidoCompra(_context, _mapper);
```

! Erros em produção são invisíveis

Todos os blocos catch da API descartam a exceção silenciosamente. Não há como diagnosticar falhas sem acesso direto ao servidor.

Autenticação — SHA-256 sem salt (client-side)

A senha é convertida para SHA-256 pelo frontend antes do envio. O backend recebe o hash e compara diretamente com o valor armazenado:

```
// RepositorioAutenticacao.cs:23  
_context.Usuario.FirstOrDefault(  
x => x.Login == usuario && x.Senha == senha && x.HabilitaLogin)
```

- **Positivo:** não há armazenamento de senha em texto puro
- **Problema 1 — sem salt:** senhas iguais geram hashes idênticos — vulnerável a rainbow tables
- **Problema 2 — SHA-256 rápido:** algoritmo genérico, não projetado para proteger senhas
- **Problema 3 — hash client-side:** do ponto de vista do servidor o hash é a credencial; interceptar o hash sobre HTTP permite autenticação direta
- **Recomendação:** bcrypt ou PBKDF2 no backend após receber a senha em texto via HTTPS

5. Banco de Dados

Modelagem

- ~50 entidades mapeadas com IEntityConfiguration — estrutura adequada
- Relacionamentos via Fluent API — positivo
- LazyLoading desabilitado — correto

Problemas Estruturais

! Senhas armazenadas como SHA-256 sem salt

O banco armazena o hash SHA-256 gerado pelo frontend. Não é texto puro, mas SHA-256 sem salt é vulnerável a rainbow tables e ataques de dicionário. Senhas comuns podem ser revertidas. bcrypt/Argon2 com salt único por usuário são os algoritmos corretos para armazenamento de senhas.

! Modelo de permissões — 80+ colunas boolean na entidade Usuario

Permissões como colunas boolean impossibilitam perfis de acesso e reatribuição em massa.

Versionamento

- **Sem migrations do EF** — schema alterado manualmente
- Script WHEngenharia.sql não versionado nem automatizado

6. Segurança

! CRÍTICO — Credenciais expostas no repositório

Múltiplas credenciais de produção estão commitadas no repositório.

Arquivo: appsettings.Development.json / appsettings.Test.json

Código: String de conexão ao SQL Server de produção

Impacto: Acesso direto ao banco de dados de produção

Arquivo: ServicoEmail.cs:35

Código: smtp.Credentials = new NetworkCredential("osvaldo.torezan@solinski.com.br",
"#jA8AcZE8EW%UhG")

Impacto: Comprometimento da conta de e-mail corporativa

Arquivo: appsettings.json

Código: "Key": "0D3A!F47aAchC68n4E\$9rB43j2&51%C1Dz@!0u558A24"

Impacto: Qualquer pessoa pode forjar tokens JWT e autenticar como qualquer usuário

Outros Problemas Críticos

! HTTP sem TLS

UseHttpsRedirection está desabilitado. config.js aponta para http:// em produção. Dados financeiros e hashes SHA-256 de senha trafegam sem criptografia. Capturar o hash equivale a capturar a credencial de acesso.

! Hash SHA-256 de senha trafega como query string

/api/Usuario/AlterarSenha?id=1&senha;=HASH — o hash (credencial efetiva) fica exposto em logs de servidor, proxies e histórico do navegador.

! Endpoints financeiros sem autenticação

[AllowAnonymous] em GeraPedidosInternos, GeraPrevisaoFluxoCaixa e AjustaDataFaturamento.

! CORS completamente aberto

SetIsOriginAllowed(origin => true) sobrescreve a configuração de CorsOrigins.

! Infraestrutura em conta pessoal do desenvolvedor

BD hospedado em conta Azure pessoal. A empresa não tem controle garantido.

7. Performance

! Carregamento sem paginação

RepositorioMaterial.Get() retorna todos os registros sem limite.

! Dupla consulta para busca

BuscaMaterialPorTexto executa 2 queries e dedução em memória.

! Cálculo de saldo em memória

CalculaSaldoDoMes carrega todos os registros do mês e itera via LINQ.

! DEFs hardcoded

60+ códigos hardcoded no C#; qualquer mudança exige deploy.

! Sem cache

Dados de referência buscados do banco em cada requisição.

8. DevOps e Processo

! Pipeline de Deploy: Inexistente

Deploy manual, sem scripts, sem workflows, sem documentação.

- **Ausência de .gitignore** adequado — credenciais foram commitadas
- Sem tags de release ou controle de versão semântico

Item	Status
Testes unitários	Ausente
Testes de integração	Ausente
Pipeline CI	Ausente
Pipeline CD	Ausente
Logging centralizado	Ausente
Documentação de API (Swagger)	Presente
Documentação de deploy	Ausente
Migrations EF	Ausente

9. Problemas Críticos (ALTA PRIORIDADE)

Ordenados por impacto:

#	Problema	Impacto	Arquivo/Local
1	Credenciais de BD no repositório	Acesso direto ao banco de produção	appsettings.Development.json
2	Senha SMTP hardcoded	Comprometimento de conta de e-mail	ServicoEmail.cs:35
3	JWT signing key no repositório	Forge de tokens — acesso total ao sistema	appsettings.json
4	Senhas SHA-256 sem salt (BD)	Hashes vulneráveis a rainbow tables; SHA-256 sem salt/bcrypt	Login.vue:122 + RepositorioAutenticacao.cs:23
5	Hash SHA-256 trafega como query string	Hash (= credencial) exposto em logs e proxies	UsuarioController.cs:131
6	Endpoints financeiros sem auth	Qualquer pessoa executa operações financeiras	ServicesController.cs:36, 52, 68
7	Bug: cálculo duplicado FluxoCaixa	DEF 03.33 somado 3x — relatórios incorretos	ServicoFluxoCaixa.cs:85, 88, 90
8	HTTP sem TLS	Dados e hashes sem criptografia	Program.cs + config.js
9	Infra em conta pessoal	Risco de descontinuidade	Azure pessoal
10	Zero logging de erros	Falhas invisíveis em produção	Todos os controllers

10. Melhorias Recomendadas

Imediato — Segurança

- 1. Rotacionar TODAS as credenciais expostas — BD, SMTP, JWT key.
- 2. Transferir infraestrutura Azure para conta corporativa da empresa.
- 3. Remover credenciais do código — usar Azure Key Vault ou variáveis de ambiente.
- 4. Substituir SHA-256 por bcrypt/PBKDF2 no backend: receber senha via HTTPS (body), aplicar hash com salt no servidor, eliminar js-sha256 do frontend.
- 5. Habilitar HTTPS obrigatório — habilitar UseHttpsRedirection e configurar SSL.
- 6. Corrigir AlterarSenha: receber senha no body (POST), não query string.
- 7. Proteger endpoints do ServicesController com autenticação.
- 8. Corrigir bug do FluxoCaixa — dupla inclusão do DEF 03.33.

Curto Prazo (3-6 meses)

- 1. Implementar pipeline CI/CD (GitHub Actions).
- 2. Adicionar logging centralizado — Serilog ou Application Insights.
- 3. Refatorar injeção de dependência — registrar serviços no Startup.cs.
- 4. Adicionar testes unitários — priorizar ServicoFluxoCaixa.

Médio Prazo (6-12 meses)

- 1. Migrar para Vue 3 — Vue 2 está em EOL.
- 2. Implementar RBAC — substituir 80+ colunas boolean por modelo de roles.
- 3. Implementar EF Migrations para rastrear mudanças de schema.
- 4. Fragmentar api.service.js por domínio.

11. Classificação Final do Sistema



ALTO RISCO

Justificativa

O uso de SHA-256 no frontend é positivo em relação a texto puro, mas não elimina os riscos de segurança. Os problemas restantes são de natureza **crítica e ativa**:

! Credenciais de produção no repositório

Acesso direto ao banco de dados de produção para qualquer pessoa com acesso ao repositório.

! JWT signing key exposta

Permite autenticação como qualquer usuário sem precisar de senha.

! SHA-256 sem salt (client-side)

Rainbow tables podem reverter hashes comuns. Hash no cliente significa que o hash é a credencial efetiva — interceptar o tráfego HTTP exprime acesso completo.

! Bug no Fluxo de Caixa

Relatórios financeiros com valores incorretos.

! Dependência total do desenvolvedor atual

Infraestrutura em conta pessoal, sem documentação de deploy.

Relatório gerado com base na análise do código-fonte. Abril de 2026.